PRACTICAL-5

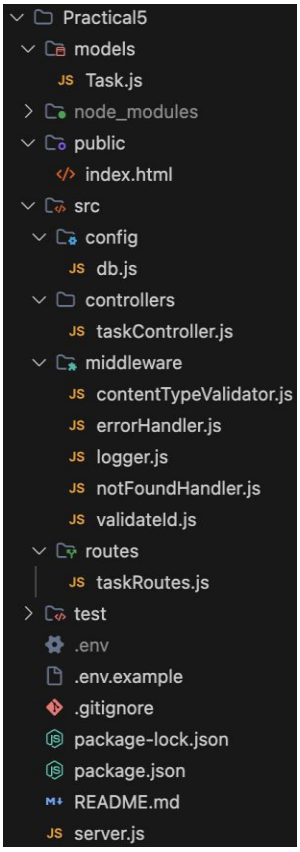
PROBLEM DEFINITION:

• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.
• Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).
• Replace the in-memory array from Practical 4 with real Mongoose model operations.
• Test all CRUD operations against the live database using Postman.
• Ensure validation errors are returned as structured JSON, not raw Mongoose error objects.

SUPPLEMENTARY PROBLEMS:

• Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high'].
• Add a pre-save hook that automatically trims whitespace from the title field.
• Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.

ARCHITECTURE / DIAGRAM:

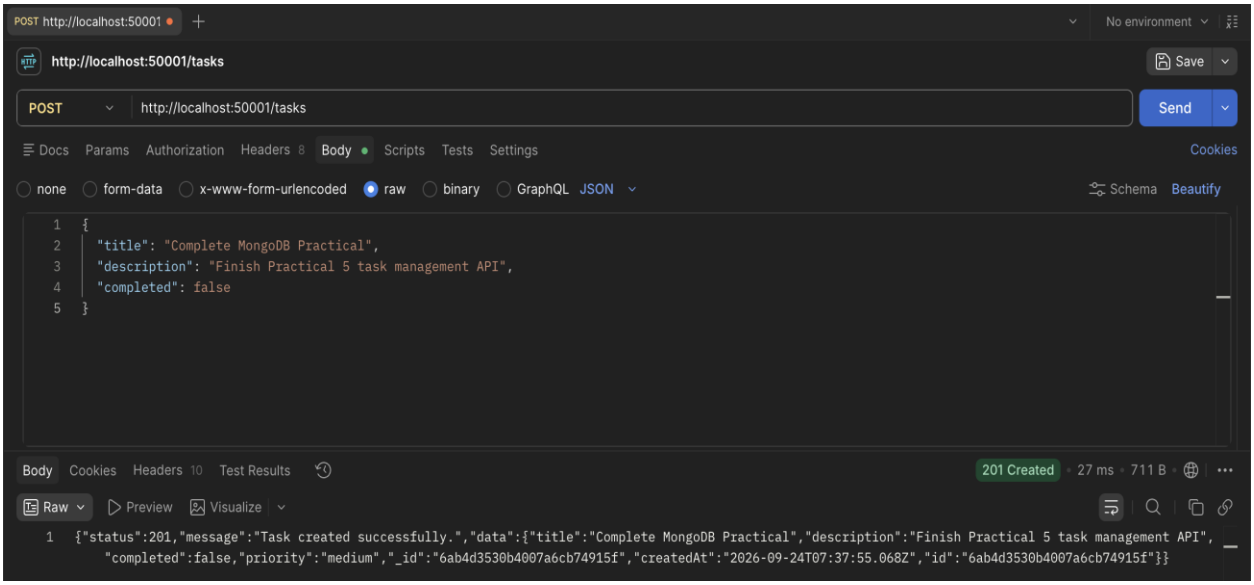


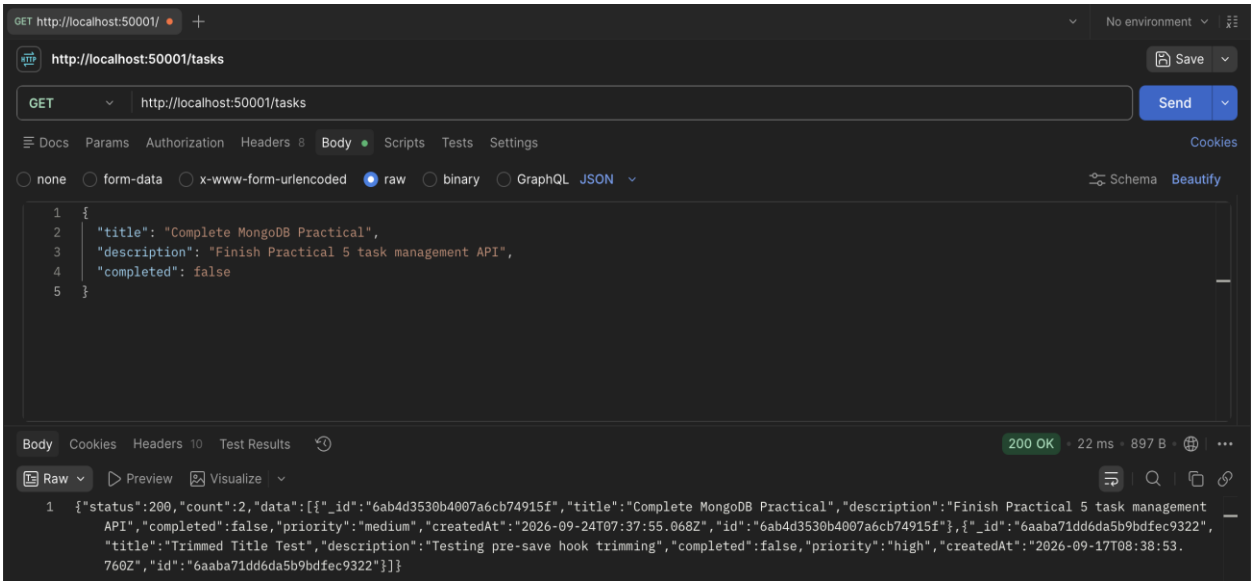
OUTPUT (SUPPLEMENTARY PROBLEMS INCLUDED):

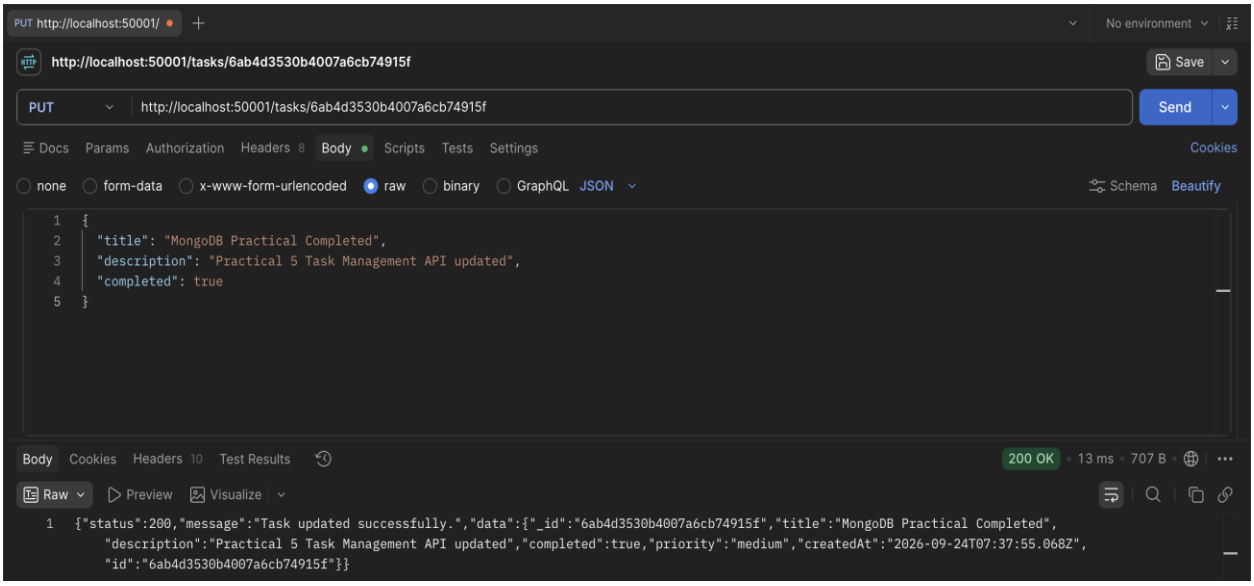
```
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practicals % cd Practical5
kamanijainamrajeshkumar@Kamanis-MacBook-Air Practical5 % npm start

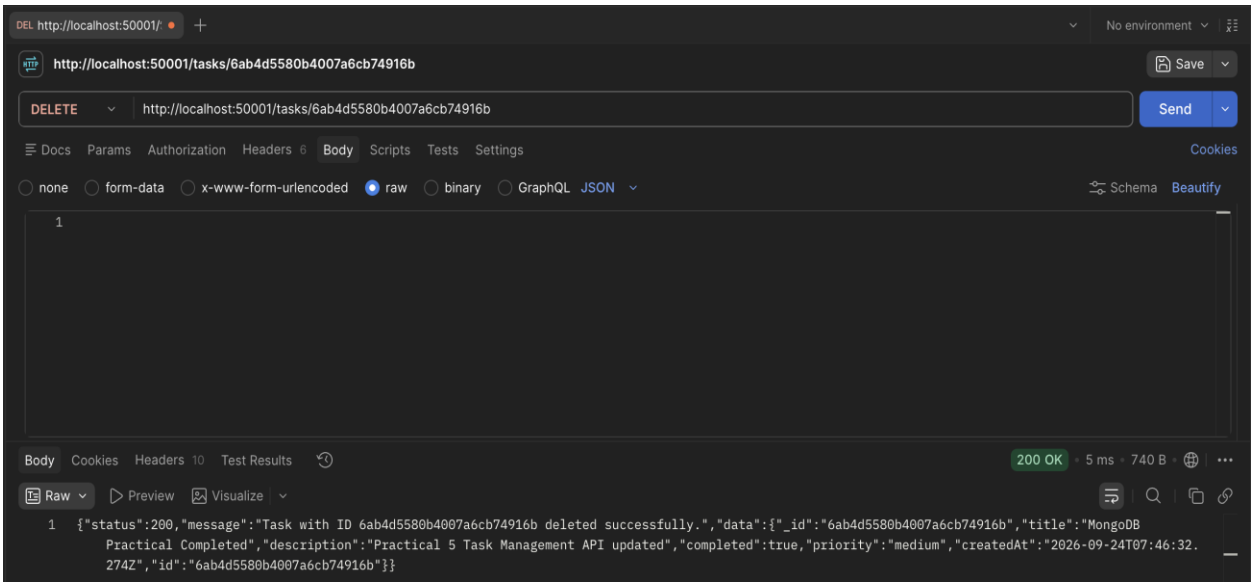

> task-manager-mongoose-api@1.0.0 start
> node server.js

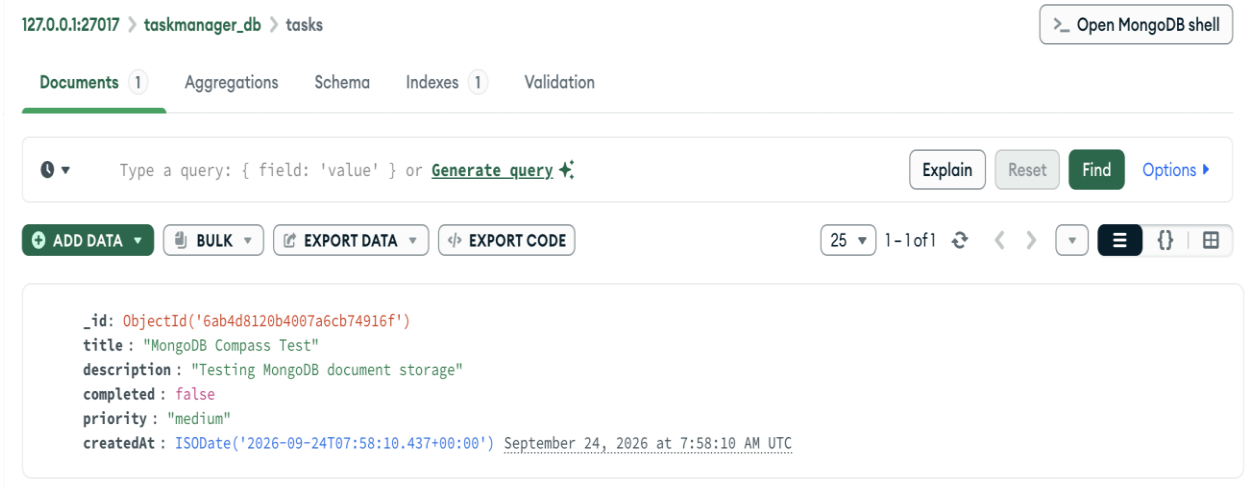
MongoDB connected
🌿 Connected to MongoDB: 127.0.0.1/taskmanager_db
[WARN] Port 5000 is in use. Retrying on port 50001...
MongoDB connected
🌿 Connected to MongoDB: 127.0.0.1/taskmanager_db
Server running on port 50001.
=====================================================
🚀 Practical 5: Task API (Mongoose & MongoDB) running on port 50001
🌐 Visual API Dashboard: http://localhost:50001
📡 Base API Endpoint:    http://localhost:50001/tasks
=====================================================
```

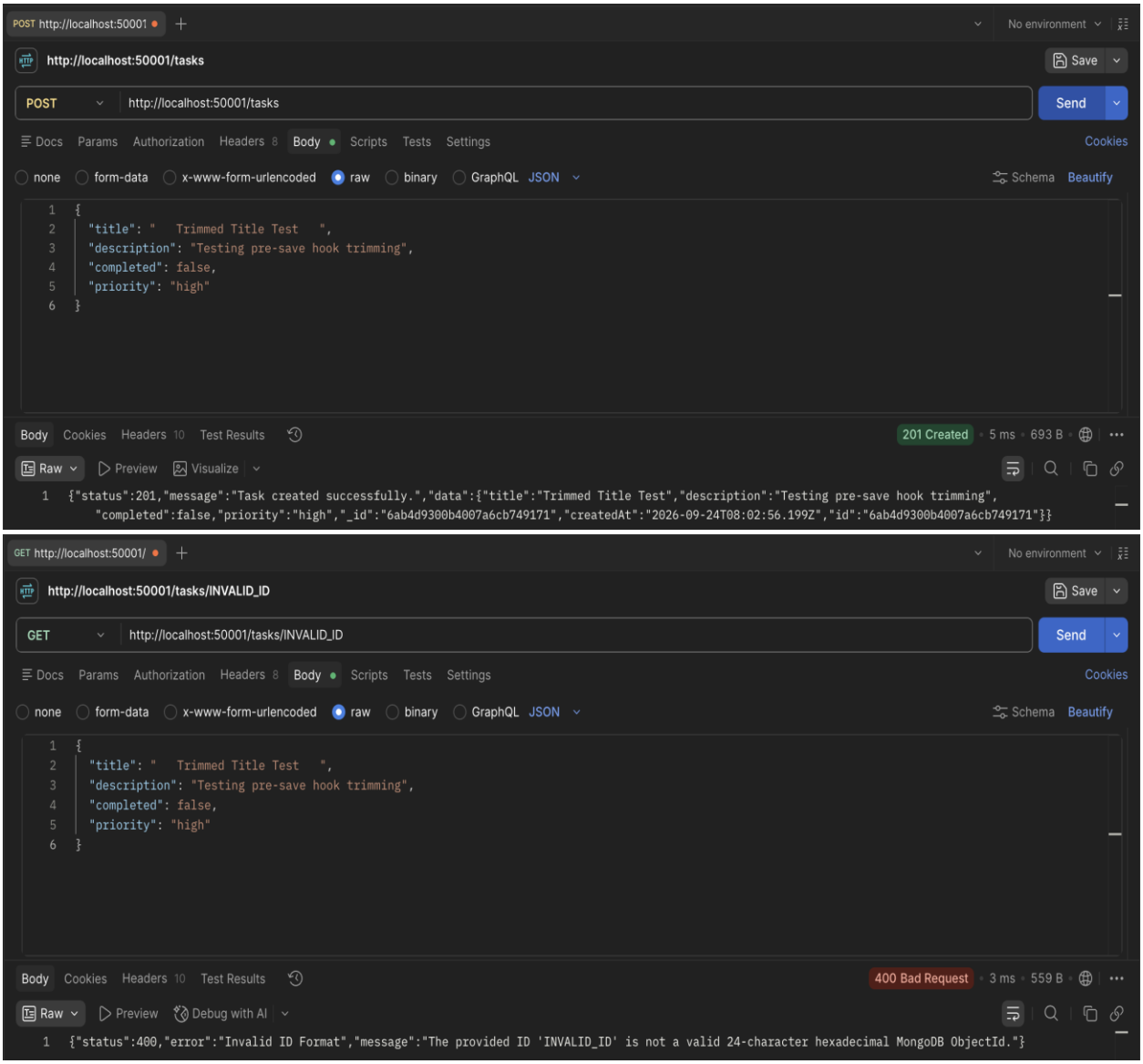





PUT http://localhost:50001/
No environment
http://localhost:50001/tasks/6ab4d3530b4007a6cb74915f
Save
PUT
http://localhost:50001/tasks/6ab4d3530b4007a6cb74915f
Send
Docs Params Authorization Headers 8 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
{
"title": "MongoDB Practical Completed",
"description": "Practical 5 Task Management API updated",
"completed": true
}
Body Cookies Headers 10 Test Results
200 OK 13 ms 707 B
Raw Preview Visualize
{"status":200,"message":"Task updated successfully.","data":{"_id":"6ab4d3530b4007a6cb74915f","title":"MongoDB Practical Completed",
"description":"Practical 5 Task Management API updated","completed":true,"priority":"medium","createdAt":"2026-09-24T07:37:55.068Z",
"id":"6ab4d3530b4007a6cb74915f"}}


DEL http://localhost:50001/
No environment
http://localhost:50001/tasks/6ab4d5580b4007a6cb74916b
Save
DELETE
http://localhost:50001/tasks/6ab4d5580b4007a6cb74916b
Send
Docs Params Authorization Headers 6 Body Scripts Tests Settings
Cookies
none form-data x-www-form-urlencoded raw binary GraphQL JSON
Schema Beautify
Body Cookies Headers 10 Test Results
200 OK 5 ms 740 B
Raw Preview Visualize
{"status":200,"message":"Task with ID 6ab4d5580b4007a6cb74916b deleted successfully.","data":{"_id":"6ab4d5580b4007a6cb74916b","title":"MongoDB
Practical Completed","description":"Practical 5 Task Management API updated","completed":true,"priority":"medium","createdAt":"2026-09-24T07:46:32.
274Z","id":"6ab4d5580b4007a6cb74916b"}}


127.0.0.1:27017 > taskmanager_db > tasks
Open MongoDB shell
Documents 1
Aggregations
Schema
Indexes 1
Validation
Type a query: { field: 'value' } or Generate query
Explain
Reset
Find
Options
ADD DATA
BULK
EXPORT DATA
EXPORT CODE
25
1-1 of 1
_id: ObjectId('6ab4d8120b4007a6cb74916f')
title : "MongoDB Compass Test"
description : "Testing MongoDB document storage"
completed : false
priority : "medium"
createdAt : ISODate('2026-09-24T07:58:10.437+00:00') September 24, 2026 at 7:58:10 AM UTC



KEY QUESTIONS / ANALYSIS:

1) What is the purpose of a schema in a NoSQL database like MongoDB, given that MongoDB itself is schema-less?

A schema defines the structure of documents, including field names, data types, required fields, and default values. Mongoose uses schemas to validate data before saving it to MongoDB, ensuring data consistency and preventing invalid data. It also makes the application easier to maintain by keeping data structures consistent. Schemas help developers detect errors early and ensure that documents follow the expected format.

2) Why is it important to define required fields and default values at the schema level rather than relying on frontend validation alone?

Schema-level validation ensures that data remains valid even when requests come from different clients, such as Postman or other applications. Frontend validation can be bypassed, but Mongoose validates data on the server before saving it to the database. Default values also ensure that fields have appropriate values when they are not provided.

3) What happens internally when a document fails Mongoose validation? Where is the request stopped?

When a document fails Mongoose validation, Mongoose generates a validation error before saving the document to MongoDB. The error is passed to the Express error-handling code, which returns a structured JSON response to the client. The invalid document is not saved to the database.

GITHUB REPOSITORY:

https://github.com/9081495207/ITUE301-Advanced-Web-Development-Frameworks